Production-Grade AI Commerce Microservices Platform

Built with PySpark Data Engineering, FastAPI Microservices, LLM-Based Semantic Search & Production Observability

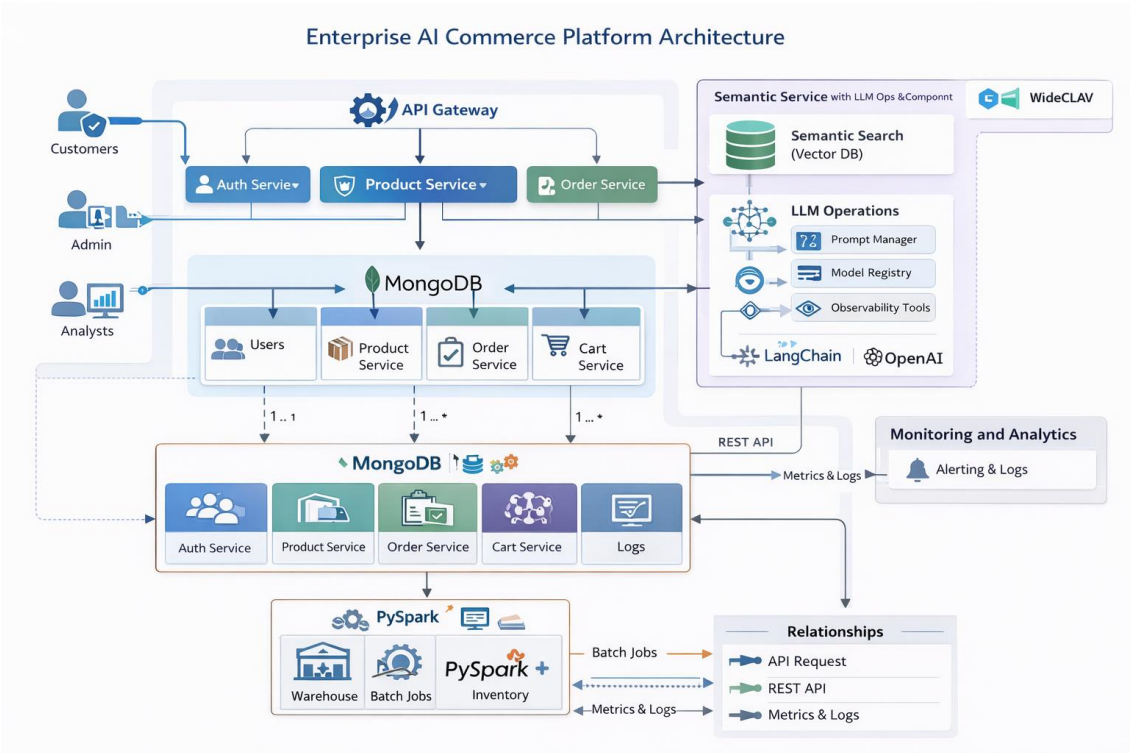
1. Problem Statement

A fashion e-commerce company maintains its inventory stock data in remote storage while product, cart, and user data are handled via application services. The organization wants to modernize into an AI-ready architecture.

Participants must build a cloud-native microservices platform that ingests ONLY inventory data using PySpark from remote storage, processes it, and loads it into MongoDB. Product browsing, authentication, cart, and order workflows must be built as independent microservices.

Later, the system must evolve by implementing semantic search and full LLM Ops practices including embedding lifecycle management, model versioning, monitoring, and redeployment.

System Architecture Diagram



Mile stone 1	**40 marks**
Milestone 2	**100 marks**
Milestone 3	**80 marks**
Milestone4	**100 marks**
Total	**320 marks**

Milestone 1 – Inventory Data Engineering with PySpark

Total Marks: 40

Tasks (40 Marks Each):

1. Connect to remote storage and load inventory dataset using PySpark.
2. Perform data cleansing and schema validation.
3. Transform dataset to structured MongoDB schema.
4. Load processed inventory data into MongoDB.

Expected Deliverables:

- PySpark ingestion scripts.
- Validated MongoDB inventory collection. (Generate synthesis data)
- Schema validation documentation.
- Execution logs of data pipeline.

MongoDB Logical Schema - Enterprise AI Commerce Platform

USERS

Column Name
user_id (Primary Key)
first_name
last_name
email (Unique)
password_hash
Role
preferences

personalization_vector
created_at

PRODUCTS

Column Name
product_id (Primary Key)
Title
description
category
Price
Rating
embedding_vector
created_at

INVENTORY (Loaded via PySpark)

Column Name
inventory_id (Primary Key)
product_id (Foreign Key → PRODUCTS)
warehouse_id
available_stock
reserved_stock
damaged_stock
reorder_level
last_updated
batch_number

CARTS

Column Name
cart_id (Primary Key)
user_id (Foreign Key → USERS)
updated_at

CART_ITEMS

Column Name
cart_item_id (Primary Key)
cart_id (Foreign Key → CARTS)
product_id (Foreign Key → PRODUCTS)
quantity
unit_price

ORDERS

Column Name
order_id (Primary Key)
user_id (Foreign Key → USERS)
total_amount
payment_status
order_status

payment_reference
order_date

ORDER_ITEMS

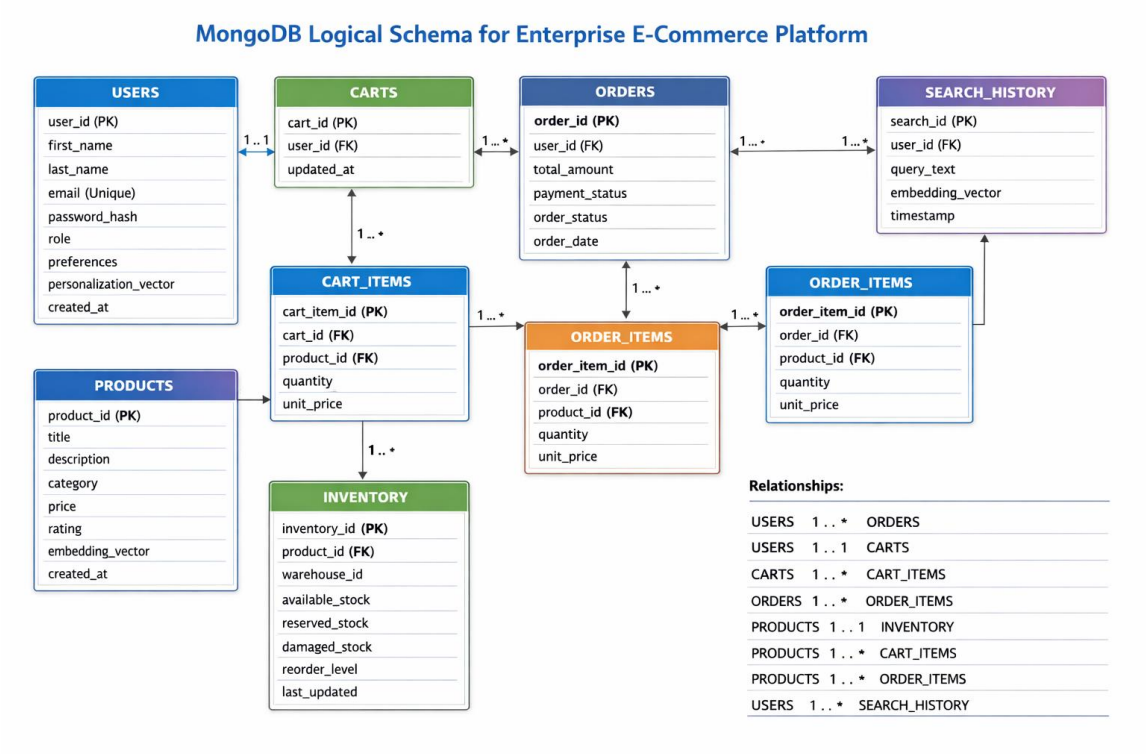
Column Name
order_item_id (Primary Key)
order_id (Foreign Key → ORDERS)
product_id (Foreign Key → PRODUCTS)
quantity
unit_price

SEARCH_HISTORY (Semantic & LLM Operations)

Column Name
search_id (Primary Key)
user_id (Foreign Key → USERS)
query_text
embedding_vector
model_version
timestamp

Relationships

- USERS (1) → (M) ORDERS
- USERS (1) → (1) CARTS
- CARTS (1) → (M) CART_ITEMS
- ORDERS (1) → (M) ORDER_ITEMS
- PRODUCTS (1) → (1) INVENTORY
- PRODUCTS (1) → (M) CART_ITEMS
- PRODUCTS (1) → (M) ORDER_ITEMS
- USERS (1) → (M) SEARCH_HISTORY



-

Milestone 2 – Core Microservices (Auth, Product, Cart, Order)

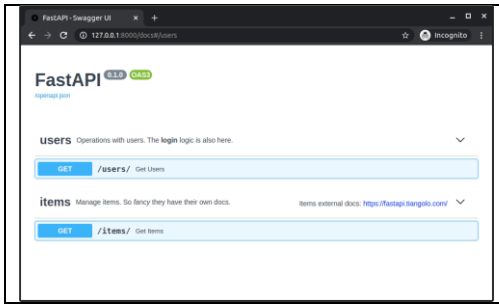
Total Marks: 100

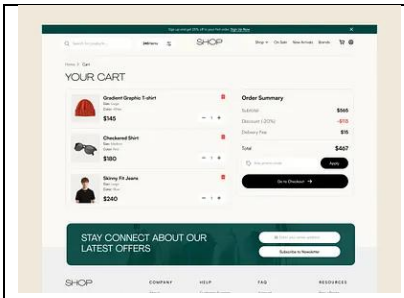
Tasks (25 Marks Each):

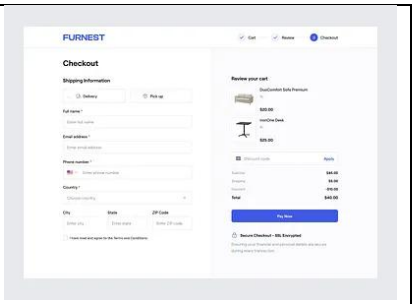
5. Implement Authentication Service with login interface.
6. Build Product Service integrated with inventory stock.
7. Build Cart Service with stock validation.
8. Build Order Service with mock payment gateway.

Expected Deliverables:

- Microservice folders with independent FastAPI apps.
- API documentation.
- Dockerfiles for services.
- Demonstration of end-to-end checkout flow.







Milestone 3 – Semantic Search Implementation

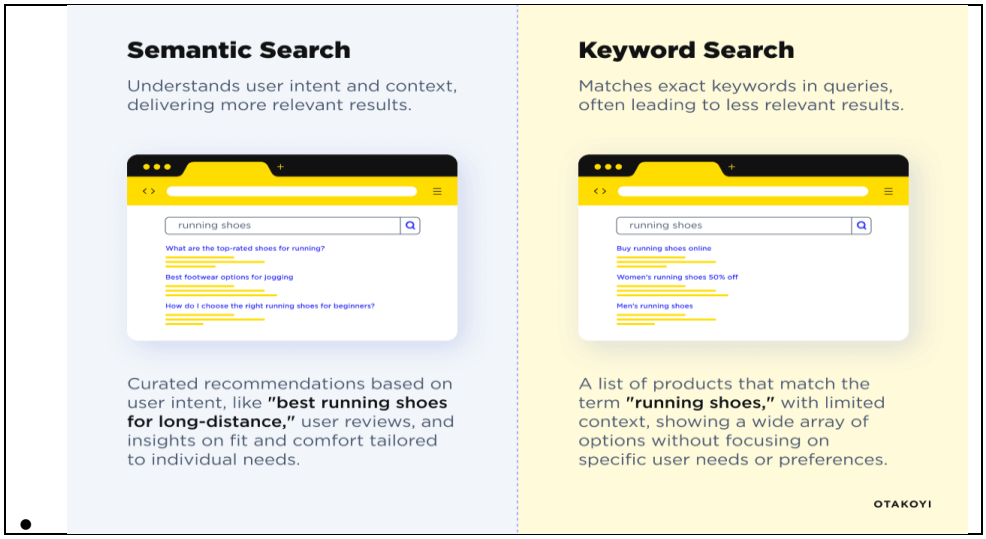
Total Marks: 80

Tasks (80 Marks Each):

9. Generate product embeddings and store them.
10. Implement cosine similarity search API.
11. Integrate semantic search into Product Service.

Expected Deliverables:

- Semantic search endpoint demo.
- Embedding generation script.
- Similarity logic implementation.
- Deployment proof after semantic update.
-



Milestone 4 – LLM Ops & Production Monitoring

Total Marks: 100

Tasks (25 Marks Each):

12. Implement model versioning and embedding version tracking.
13. Create logging for prompts, responses, and search queries.
14. Integrate monitoring using Application Insights or structured logging.
15. Perform controlled redeployment after LLM/embedding update.

Expected Deliverables:

- LLM version tracking documentation.
- Monitoring dashboard screenshots.
- Structured logging implementation.
- Demonstration of redeployment workflow after model update.